

Name: Erik Schreider

Challenge 4: Data Integration and Analytical Queries

Tasks:

1. Analogous to the other workbook (Challenge 3).
2. Again, please put your SQL commands in code blocks. Be careful that long queries are not cut off at the right border of the page.

Title

Space Traffic Analysis - Kollisionsrisiken und Nachhaltigkeit im Erdorbit

Data Transformation

How did you perform your data transformation? Fill out the following list with short answers and optionally one example SQL query each. Which tasks were necessary for your project? Why? Why not? How did you do it? Write at least one SQL query here. (Ex. Sheet 4, Exercise 1)

- Checking data quality and fixing data errors:

Das ETL prüft Pflichtfelder bereits beim Parsen. CelesTrak-Zeilen ohne **NORAD_CAT_ID**, **EPOCH**, **INCLINATION**, **ECCENTRICITY** oder **MEAN_MOTION** werden nicht in die Staging-CSV geschrieben. Im Warehouse-Load werden außerdem nur Zeilen mit fachlichem Schlüssel weiterverarbeitet.

```
SELECT
    "source_group",
    COUNT(*) AS invalid_rows
FROM "stg_celestrak_objects"
WHERE "norad_id" IS NULL
    OR "epoch" IS NULL
    OR "source_group" IS NULL
GROUP BY "source_group";
```

- Harmonization / Normalization:

CelesTrak liefert Orbitdaten pro Objekt und Quellgruppe. UCS liefert beschreibende Satelliten-Metadaten. Beide Quellen werden über die stabile NORAD-ID harmonisiert. Operatoren werden in **dim_operator** normalisiert. Orbitklassen wie **LEO**, **MEO**, **GEO** und **HEO** werden auf **dim_orbit_band** abgebildet. Fehlende Textwerte werden fachlich als **UNKNOWN** behandelt.

```
SELECT
    c."norad_id",
```

```

MAX(c."object_name") AS object_name,
MAX(u."operator_name") AS operator_name,
MAX(u."orbit_type") AS orbit_type
FROM "stg_celestrak_objects" c
LEFT JOIN "stg_ucs_satellites" u
    ON c."norad_id" = u."norad_id"
WHERE c."norad_id" IS NOT NULL
GROUP BY c."norad_id";

```

- Deduplication:

Deduplication ist notwendig, weil CelesTrak dasselbe Objekt in mehreren Gruppen enthalten kann und der ETL wiederholt läuft. CelesTrak wird mit **NORAD_ID + EPOCH + SOURCE_GROUP** unterschieden. UCS wird über **NORAD_ID** dedupliziert. Launches werden über **LAUNCH_ID** dedupliziert; falls keine stabile ID vorhanden ist, erzeugt der ETL ersatzweise eine ID aus Launch-Name, Launch-Datum und Provider.

```

SELECT
    "norad_id",
    "epoch",
    "source_group",
    COUNT(*) AS duplicate_count
FROM "stg_celestrak_objects"
GROUP BY
    "norad_id",
    "epoch",
    "source_group"
HAVING COUNT(*) > 1;

```

- Fuzzy entity matching (Levensthein, Soundex, ...):

Fuzzy Matching wurde bewusst nicht verwendet. Bei Satelliten ist die NORAD-ID der fachlich stabile Integrationsschlüssel. Namen wie **STARLINK-1007** können zwischen Quellen leicht variieren, während die NORAD-ID eindeutig ist. Ein Levenshtein-Matching auf Namen würde hier mehr Fehlzuordnungen riskieren als Nutzen bringen.

- Data Fusion (merge multiple rows into one target row):

Data Fusion findet beim Aufbau von **dim_space_object** statt. CelesTrak liefert Objektname, Orbitdaten und Quellgruppe. UCS liefert Betreiber, Land, Zweck, Orbitklasse, Status und Startdatum. Diese Informationen werden je NORAD-ID zu einem Space-Object-Dimensionsdatensatz fusioniert.

```

SELECT
    c."norad_id",
    MAX(c."object_name") AS object_name,
    MAX(u."purpose") AS purpose,
    MAX(u."operational_status") AS operational_status,
    MAX(u."launch_date") AS launch_date
FROM "stg_celestrak_objects" c
LEFT JOIN "stg_ucs_satellites" u

```

```
ON c."norad_id" = u."norad_id"  
GROUP BY c."norad_id";
```

Data Integration

Write down a *MERGE* command to integrate data from your staging area into your target data-warehouse schema: (Ex. Sheet 4, Exercise 2)

```
MERGE INTO "dim_space_object" target  
USING (  
  SELECT  
    c."norad_id" AS "object_id",  
    c."norad_id",  
    MAX(c."object_name") AS "object_name",  
    CASE  
      WHEN MAX(c."source_group") = 'debris'  
      THEN 'DEBRIS'  
      ELSE 'SATELLITE'  
    END AS "object_type",  
    MAX(u."operational_status") AS "operational_status",  
    MAX(u."purpose") AS "purpose",  
    MAX(u."launch_date") AS "launch_date",  
    MAX(o."operator_id") AS "operator_id"  
  FROM (  
    SELECT  
      "norad_id",  
      MAX("object_name") AS "object_name",  
      MAX("source_group") AS "source_group"  
    FROM "stg_celestrak_objects"  
    WHERE "norad_id" IS NOT NULL  
    GROUP BY "norad_id"  
  ) c  
  LEFT JOIN "stg_ucs_satellites" u  
    ON c."norad_id" = u."norad_id"  
  LEFT JOIN "dim_operator" o  
    ON COALESCE(u."operator_name", 'UNKNOWN') = o."operator_name"  
  GROUP BY c."norad_id"  
) source  
ON target."norad_id" = source."norad_id"  
WHEN MATCHED THEN UPDATE SET  
  target."object_name" = source."object_name",  
  target."object_type" = source."object_type",  
  target."operational_status" = source."operational_status",  
  target."purpose" = source."purpose",  
  target."launch_date" = source."launch_date",  
  target."operator_id" = source."operator_id"  
WHEN NOT MATCHED THEN INSERT  
(  
  "object_id",  
  "norad_id",  
  "object_name",
```

```

        "object_type",
        "operational_status",
        "purpose",
        "launch_date",
        "operator_id"
    )
VALUES
(
    source."object_id",
    source."norad_id",
    source."object_name",
    source."object_type",
    source."operational_status",
    source."purpose",
    source."launch_date",
    source."operator_id"
);

```

Analytical Queries

Write 7 SQL queries here. These can be your query ideas from your presentation (Challenge 1), but can also be other queries. Write at least one query with **GROUPING SETS** (or **ROLLUP** or **CUBE**), one with a window function (no ranking), one with a ranking function, one with a statistical function (e.g., **STDEV_POP**), and one skyline query. Start each query with a comment that describes the query. (Ex. Sheet 6)

```

-- Query 1: GROUPING SETS / ROLLUP / CUBE
-- Counts objects by orbit class and object type, including subtotals
-- per orbit class and a grand total.
SELECT
    COALESCE(b."orbit_class", 'ALL_ORBITS') AS orbit_class,
    COALESCE(o."object_type", 'ALL_OBJECT_TYPES') AS object_type,
    SUM(f."object_count") AS object_count,
    AVG(f."debris_risk_score") AS avg_debris_risk_score
FROM "fact_object_position_observation" f
LEFT JOIN "dim_orbit_band" b
    ON f."orbit_band_id" = b."orbit_band_id"
LEFT JOIN "dim_space_object" o
    ON f."object_id" = o."object_id"
GROUP BY GROUPING SETS
(
    (b."orbit_class", o."object_type"),
    (b."orbit_class"),
    ()
)
ORDER BY object_count DESC;

```

```

-- Query 2: Window Function (no ranking)
-- Shows the daily object observations and a 7-day moving average.
WITH daily_counts AS

```

```
(
  SELECT
    CAST(f."observation_timestamp" AS DATE) AS observation_date,
    SUM(f."object_count") AS object_count
  FROM "fact_object_position_observation" f
  WHERE f."observation_timestamp" IS NOT NULL
  GROUP BY CAST(f."observation_timestamp" AS DATE)
)
SELECT
  observation_date,
  object_count,
  AVG(object_count) OVER
  (
    ORDER BY observation_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7_days
FROM daily_counts
ORDER BY observation_date;
```

```
-- Query 3: Window Function (ranking query)
-- Ranks operators by the number of observed objects.
WITH operator_counts AS
(
  SELECT
    COALESCE(op."operator_name", 'UNKNOWN') AS operator_name,
    COALESCE(op."country", 'UNKNOWN') AS country,
    SUM(f."object_count") AS object_count,
    AVG(f."debris_risk_score") AS avg_debris_risk_score
  FROM "fact_object_position_observation" f
  JOIN "dim_space_object" o
    ON f."object_id" = o."object_id"
  LEFT JOIN "dim_operator" op
    ON o."operator_id" = op."operator_id"
  GROUP BY
    COALESCE(op."operator_name", 'UNKNOWN'),
    COALESCE(op."country", 'UNKNOWN')
)
SELECT
  operator_name,
  country,
  object_count,
  avg_debris_risk_score,
  DENSE_RANK() OVER (ORDER BY object_count DESC) AS operator_rank
FROM operator_counts
ORDER BY operator_rank;
```

```
-- Query 4: Statistical Function
-- Calculates statistical orbit values per orbit class.
SELECT
```

```
COALESCE(b."orbit_class", 'UNKNOWN') AS orbit_class,  
SUM(f."object_count") AS object_count,  
AVG(f."inclination_deg") AS avg_inclination_deg,  
STDDEV_POP(f."inclination_deg") AS stddev_inclination_deg,  
AVG(f."debris_risk_score") AS avg_debris_risk_score  
FROM "fact_object_position_observation" f  
LEFT JOIN "dim_orbit_band" b  
  ON f."orbit_band_id" = b."orbit_band_id"  
GROUP BY COALESCE(b."orbit_class", 'UNKNOWN')  
ORDER BY object_count DESC;
```

```
-- Query 5: Skyline Query
-- Finds orbit/operator combinations that are not dominated by another
-- combination with both at least as many objects and at least as much
-- average debris risk.
WITH candidates AS
(
    SELECT
        COALESCE(b."orbit_class", 'UNKNOWN') AS orbit_class,
        COALESCE(op."operator_name", 'UNKNOWN') AS operator_name,
        SUM(f."object_count") AS object_count,
        COALESCE(AVG(f."debris_risk_score"), 0) AS avg_risk
    FROM "fact_object_position_observation" f
    LEFT JOIN "dim_orbit_band" b
        ON f."orbit_band_id" = b."orbit_band_id"
    JOIN "dim_space_object" o
        ON f."object_id" = o."object_id"
    LEFT JOIN "dim_operator" op
        ON o."operator_id" = op."operator_id"
    GROUP BY
        COALESCE(b."orbit_class", 'UNKNOWN'),
        COALESCE(op."operator_name", 'UNKNOWN')
)
SELECT
    c.orbit_class,
    c.operator_name,
    c.object_count,
    c.avg_risk
FROM candidates c
WHERE NOT EXISTS
(
    SELECT 1
    FROM candidates other
    WHERE other.object_count >= c.object_count
        AND other.avg_risk >= c.avg_risk
        AND (
            other.object_count > c.object_count
            OR other.avg_risk > c.avg_risk
        )
)
ORDER BY c.avg_risk DESC, c.object_count DESC;
```

```
-- Query 6:
-- Shows the distribution of satellite purposes.
SELECT
    COALESCE(o."purpose", 'UNKNOWN') AS purpose,
    SUM(f."object_count") AS object_count,
    AVG(f."debris_risk_score") AS avg_debris_risk_score
FROM "fact_object_position_observation" f
JOIN "dim_space_object" o
    ON f."object_id" = o."object_id"
```

```
GROUP BY COALESCE(o."purpose", 'UNKNOWN')
ORDER BY object_count DESC;
```

```
-- Query 7:
-- Analyzes launch activity by provider and launch status.
SELECT
    COALESCE(l."provider_name", 'UNKNOWN') AS provider_name,
    COALESCE(l."launch_status", 'UNKNOWN') AS launch_status,
    COUNT(*) AS launch_count,
    SUM(COALESCE(l."payload_count", 0)) AS payload_count,
    SUM(COALESCE(l."success_count", 0)) AS success_count
FROM "fact_launch_event" l
GROUP BY
    COALESCE(l."provider_name", 'UNKNOWN'),
    COALESCE(l."launch_status", 'UNKNOWN')
ORDER BY launch_count DESC;
```

Please check: Have you written your name on the very top?